Inhabited Parsing

A certified parser for HTTP/1.1 chunked transfer coding

The value-dependent grammar: *the size value on the wire decides how many octets the parser must consume next* — verified in **Rocq (Coq)**

`theories/Chunked.v` · S6 — RFC 9112 §7.1

The declarative part

A grammar is **data**; the parser is a **function**; the proof they agree is a **derivation that *is* the parse**.

For chunked transfer coding, the grammar is **value-dependent**: it reads a hex number `n`, then produces *exactly* `n` octets.

The grammar — RFC 9112 §7.1

```
chunked-body   = *chunk last-chunk CRLF
chunk          = chunk-size CRLF chunk-data CRLF
chunk-data     = 1*OCTET
last-chunk     = "0" CRLF
chunk-size     = 1*HEXDIG            -- unbounded, no machine-width lie
```

The essential asymmetry: **every chunk** has a length-prefixed payload; the **last chunk** is exactly `0 CRLF`. The decoder must walk `*chunk`, stop at the zero chunk, and then swallow the terminating `CRLF`.

The crux — `Exactly n`

The size is a **value** that the *plan* depends on. The grammar needs a combinator that consumes a bounded, value-indexed run of octets:

```
(* Spec: Exactly n s  ≅  n repetitions of s, as a list *)
| Exactly : nat -> Spec Γ N A -> Spec Γ N (list A)

(* denotation — the two rules: *)
d_exactly_nil  : denote (Exactly 0 s) y i [] y i
d_exactly_cons : denote s y i a y' j ->
                 denote (Exactly n s) y' j as y'' k ->
                 denote (Exactly (S n) s) y i (a :: as) y'' k
```

`Exactly 0` is the empty segment; `Exactly (S n)` prepends one token and recurses on `n`. This is the *only* new combinator S6 adds to the core `Spec`.

The Spec — chunks as data

```
Definition hex_natural_spec : Spec ascii unit chunked_nt nat :=
  Map (fun p => hex_digits_to_nat (fst p :: snd p))
      (Seq hex_digit_spec (Many hex_digit_spec)).

Definition data_chunk_spec : Spec ascii unit chunked_nt (list ascii) :=
  Bind hex_natural_spec (fun n : nat =>
    if n =? 0 then Fail                                  (* 0 is not a data chunk *)
    else Map (fun p => fst (snd p))
             (Seq crlf (Seq (Exactly n octet_spec) crlf))).

Definition last_chunk_spec : Spec ascii unit chunked_nt (list ascii) :=
  Bind hex_natural_spec (fun n : nat =>
    if n =? 0 then Map (fun _ => nil) crlf               (* 0 CRLF *)
    else Fail).

Definition chunked_body_spec : Spec ascii unit chunked_nt (list ascii) :=
  Map fst (Seq (Map (fun p => concat (fst p))
                    (Seq (Many data_chunk_spec) last_chunk_spec)) crlf).
```

The value `n` flows through `Bind` into `Exactly n` — the grammar's *length prefix* becomes the parser's *loop bound*.

The denotation

`denote` is a **proof-relevant relation**: a derivation *is* the certificate, and the parsed value is an index of the proof.

```
denote G S y i a y' j    (* "S, begun at i in env y, yields a at j in y'" *)
```

token type	value
`Γ`	`unit` — chunked decoding is stateless
`N`	`chunked_nt` — the **empty** nonterminal family (no recursion: a chunk is a flat value-dependent parse)
`A`	`list ascii` — the decoded payload

`token = ascii` (bytes as chars); no escapes, no chunk extensions. The `Exactly n` rules carry the length bound *in the derivation*.

The parser — value-dependent

The decoded size `n` is threaded into the consumption step:

```
parse_exactly (n : nat) (w : list ascii) : option (list ascii * list ascii) :=
  if n <=? List.length w then Some (List.firstn n w, List.skipn n w) else None.

parse_chunk (w : list ascii) : option (list ascii * list ascii) :=
  match parse_hex_size w with
  | None => None
  | Some (n, rest0) =>
      (* expect CRLF, then *)
      if n =? 0 then Some ([], rest)            (* last chunk: stop *)
      else (* parse_exactly n octets, then the trailing CRLF *)
        ...
  end.

parse_chunked_body (w : list ascii) : option (list ascii) :=
  match parse_body (3 * List.length w + 2) w with
  | Some (data, "\r" :: "\n" :: []) => Some data
  | _ => None
  end.
```

The parser is a **Fixpoint** on a fuel bound (`3·length w + 2`); each chunk decrements it. The size value `n` — not a static grammar rule — decides `parse_exactly n`.

Essence of the solution — soundness

The parser only ever produces real denotations.

```
Lemma parse_chunk_sound (prefix w data rest) :
  parse_chunk w = Some (data, rest) ->
  denote empty_grammar (prefix ++ w) chunk_spec
         tt (length prefix) data tt (length prefix + length w - length rest).

Lemma parse_chunked_body_sound (w data) :
  parse_chunked_body w = Some data ->
  denote empty_grammar w chunked_body_spec tt 0 data tt (length w).
```

Status: all `Qed.` — `char_sound` · `octet_sound` · `crlf_sound` · `parse_hex_*_sound` · `parse_exactly_sound` · `parse_chunk_sound` · `parse_body_sound` · `parse_chunked_body_sound`.

Essence of the solution — completeness

The converse is **not** a free mirror: the loose grammar `Many chunk_spec + CRLF` accepts bodies that the parser rejects (a body *without* a zero chunk). RFC 9112 mandates `*chunk last-chunk`, so the spec is **split**:

```
(* unified chunk  →  data-chunk (payload ≠ [])  ∨  last-chunk (payload = []) *)
Lemma denote_chunk_to_data / denote_chunk_to_last  (and their inverses)

Lemma parse_chunk_complete (prefix consumed rest data) :
  denote ... chunk_spec ... (length prefix) data ... (length (prefix ++ consumed)) ->
  parse_chunk (consumed ++ rest) = Some (data, rest).

Lemma parse_body_complete (fuel prefix consumed rest init) :
  denote ... (Seq (Many data_chunk_spec) last_chunk_spec) ... (init, []) ... ->
  length init < fuel ->
  parse_body fuel (consumed ++ rest) = Some (concat init, rest).

Lemma parse_chunked_body_complete (w data) :
  denote ... w chunked_body_spec ... 0 data ... (length w) ->
  parse_chunked_body w = Some data.
```

Status: all `Qed.` — `parse_hex_*_complete` (greedy extension) · `parse_exactly_complete` · `parse_chunk_complete` · `parse_body_complete` · `parse_chunked_body_complete`.

Extraction — a runnable decoder

```
From Stdlib Require Import Extraction ExtrOcamlNatBigInt ExtrOcamlChar.
Extract Constant Z.of_nat => "(fun n -> n)".
Extraction Language OCaml.
Extraction "chunked.ml" parse_chunked_body.
```

```
val parse_chunked_body : char list -> char list option   (* 497-line, self-contained *)

parse_chunked_body "5\r\nhello\r\n0\r\n\r\n"   = Some ['h';'e';'l';'l';'o']
parse_chunked_body "A\r\n0123456789\r\n0\r\n\r\n" = Some [...10 octets...]
```

`nat` → Zarith `Z.t`, `ascii` → `char`; the hex size `hex_digits_to_nat` is arbitrary-precision (no 32/64-bit size overflow).

Benchmark — extracted OCaml, Zarith

Synthetic body: `n` data chunks of `k` payload octets each, then `0 CRLF CRLF`. Decoded end-to-end; correct byte count verified.

input	chunks (×64 KB)	time	throughput
1.05 MB	16	0.11 s	9.2 MB/s
4.19 MB	64	0.40 s	10.1 MB/s
8.39 MB	128	0.78 s	10.3 MB/s
16.78 MB	256	1.57 s	10.2 MB/s
33.56 MB	512	3.09 s	10.4 MB/s
67.12 MB	1024	7.10 s	9.0 MB/s

Time doubles when input doubles → **linear**. Small 4 KB chunks: 1 MB in 0.075 s (13.4 MB/s), 16 MB in 1.24 s (12.9 MB/s) — flat too.

The fix — a single-pass `parse_exactly`

The first cut was **O(N²)**: `if n <=? length w then Some (firstn n w, skipn n w)` re-traversed the remaining suffix for every chunk.

```
(* one O(n) pass — no length / firstn / skipn over the whole suffix *)
Fixpoint parse_exactly n w :=
  match n, w with
  | O, _          => Some ([], w)
  | S n', []      => None
  | S n', c :: rest =>
      match parse_exactly n' rest with
      | None => None
      | Some (d, r) => Some (c :: d, r)
      end.
```

16 MB	naive (O(N²))	single-pass (O(N))
time	9.58 s	1.57 s
throughput	1.7 MB/s	10.2 MB/s

Thanks

Value-dependent grammar certified · **Soundness + completeness** `Qed.` · **Extraction** to a runnable OCaml decoder

`./slides/http` · `nix develop --command make` · `ocamlfind ocamlopt -package zarith -linkpkg chunked.mli chunked.ml bench_http.ml`